Software development is a continuous cycle of building new features, creating and executing tests to detect bugs, and repairing defects – repeated *ad infinitum*. This cycle must be carefully managed as software bugs or vulnerabilities can incur huge financial losses [14]. However, prior research aimed at improving software development, especially in important areas of program synthesis [3], software testing [5], and program repair [13] often depend heavily on symbolic methods, making them intractable for large systems. In addition, these techniques may require significant effort to design manual patterns or symbolic constraints, further limiting expressiveness and application scope. In contrast, the recent emergence of large language models (LLMs) with powerful generative and reasoning capabilities provides a new paradigm that can overcome the brittleness and scalability limits of traditional techniques. At the same time, program repair, test generation, and code synthesis have also become key tasks for enhancing the coding and reasoning abilities of LLMs themselves. Software is structured, complex, verifiable, and increasingly pervasive in our society. As such, software data and tasks will become a critical source for teaching LLMs code understanding, advanced planning, and ultimately contributing to the next-generation of LLM intelligence.

My research lies at the intersection of **Software Engineering** and **Artificial Intelligence**, with a central focus on synergizing software engineering principles with advanced AI technologies. Through this perspective, my work has fundamentally transformed and advanced **program repair**, **test generation**, and **program synthesis**. In program repair, I proposed **AlphaRepair** [27], an LLM-based repair technique that, for the first time, showed LLMs can directly outperform prior dedicated repair tools, and has inspired many follow-up repair work [23, 26, 29] as well as recent software agents [21, 11]. In test generation, my work like **TitanFuzz** [9] and **Fuzz4All** [24] has opened an entirely new dimension of LLM-based fuzzing which has been widely studied in software engineering, security, and other relevant fields [17]. These projects have already had real-world impact, uncovering more than 200 critical bugs or vulnerabilities in important software, including Machine Learning systems (PyTorch/TensorFlow), compilers (C/C++/Go/Java), and even quantum computing framework (Qiskit). For program synthesis, I developed **EvalPlus** [12] that shows for the first time that modern LLM performance on code generation is largely inflated due to limited testing, and has further inspired a large body of work on rigorous evaluation of LLMs for code generation. More recently, I built **Agentless** [21], an *agentless* approach to automatically solve end-to-end software development tasks, demonstrating that even a simple agentless design can outperform mainstream dedicated software agents. Agentless has been adopted by leading AI companies (e.g., OpenAI, Meta, and DeepSeek) to evaluate their recent models and to build custom scaffolds. To date, my work has attracted more than 4,800 citations, and my tools/datasets have garnered thousands of stars on GitHub and have been downloaded more than a million times. My research has received an ACM SIGSOFT Distinguished Paper Award (FSE'25), an Oral Spotlight Paper Award (COLM'24), and a CACM Research Highlight (FSE'23). Additionally, I have been supported by the prestigious Amazon AICE PhD Fellowship.

Going forward, my goal is to *revolutionize* the software development process by leveraging and optimizing LLMs and agents with software engineering techniques to assist developers in real-world software development. I intend to utilize powerful LLM agents equipped with specialized software engineering tools to solve complex and challenging tasks like synthesizing production-ready software from scratch and detecting/patching difficult security vulnerabilities. In addition, I plan to harness the vast amount of previously untapped software engineering data and rich development environments to train next-generation LLMs and agents with better coding and reasoning capabilities as well as the ability for *self-improvement*, paving the way for fully autonomous and continuously evolving LLM intelligence.

Program Repair: Reimagining Automated Debugging

Software bugs or vulnerabilities can cost companies billions in financial losses [14], and developers are estimated to spend 50% of their time debugging software systems [15]. To address this, Automated Program Repair (APR) has been extensively studied to build automatic debugging supports for developers [13]. My work has revolutionized the field of APR by harnessing the recent breakthroughs in LLMs.

Transforming APR in the LLM era. Traditional APR approaches that rely on constraint solving, code search, or simple fixing templates suffer from scalability issues and limited patch variety. To overcome these limitations, researchers have proposed learning-based APR techniques that treat repair as a neural machine translation (NMT) task, translating buggy code into correct code. However, such NMT-based APR techniques require fine-tuning on large bug-fixing datasets which can be hard to obtain and furthermore cannot generalize to new bug patterns outside of the training set. To address these issues, I developed **AlphaRepair** [27], which transforms APR from an NMT task to a *conditional generation* problem by removing the buggy code and simply asking an LLM to fill in the correct code when given bidirectional code

context. In this way, no fine-tuning or even any few-shot examples are needed, and APR can be performed in zero-shot. AlphaRepair showed for the first time that an LLM-based APR tool can achieve state-of-the-art results, outperforming all traditional tools studied for over a decade, and has pioneered the **new direction of LLM-based APR**. I also conducted the first extensive study [26] on leveraging LLMs for direct APR across different scenarios, covering nine different LLMs and five benchmarks spanning three languages. The results demonstrated the impressive performance and potential of LLM-based repair and have inspired a wave of future work along this direction [23, 20, 29] with over 600 citations. Furthermore, I built more powerful LLM-based APR inspired by the important *plastic surgery hypothesis* [4], where the fixing code snippets can already be found within the buggy project, using both fine-tuning and retrieval strategies [23].

Advancing LLM-based APR via environmental feedback. I also improved LLM-based APR by leveraging the diverse feedback signals that arise during human debugging and programming workflows. I built **Repilot** [20] to guide LLMs with a traditional Completion Engine [1] (e.g., Eclipse JDT Language Server), improving the validity of generated patches. My key insight is recognizing that LLMs generate tokens autoregressively, resembling a human writing programs, which can be guided through a Completion Engine. Repilot leverages the Completion Engine to prune infeasible tokens and proactively autocomplete suitable tokens during the repair process to produce more compilable and correct patches. Repilot has been selected as a **CACM Research Highlight**. More recently, I created **ChatRepair** [29], the first automatic *conversation-driven* APR approach that interleaves patch generation with instant environmental feedback to perform APR in a conversational style. Unlike prior LLM-based APR tools, which simply sample patches with the same input, ChatRepair learns from both environmental feedback (such as test or compilation outcomes) and previous patch attempts to generate the next patch. To date, ChatRepair has inspired a substantial body of research on iterative repair, code, and test generation powered by LLMs [6, 30]. Moreover, my insight of utilizing environment feedback to assist automated code generation[1] is now a key component widely adopted by modern software agents (e.g., Devin [8], OpenHands [19], and Replit [16]) to make autonomous decisions.

Test Generation: Modeling Heterogeneous Constraints

Detecting software bugs and vulnerabilities is critical to ensure correctness and reliability especially in widely-used systems such as software libraries or compilers. To find such bugs, developers often employ automatic testing techniques such as fuzz testing [31], also known as fuzzing, to generate large amounts of inputs to test the underlying system. My work focuses on addressing the limitations of traditional fuzzing tools by leveraging LLMs as fuzzing engines to tackle a wide range of challenging domains.

Machine Learning system fuzzing. A core component of Machine Learning (ML) systems is the use of Deep Learning (DL) libraries with thousands of APIs for building, training, and deploying models. Due to the increased adoption of ML pipelines in many critical fields, ensuring the quality of the underlying DL libraries is essential. However, traditional fuzzers struggle with complex and heterogeneous constraints (e.g., dynamic typing and numeric constraints between DL API parameters and I/O tensors) present in DL libraries, and thus are often limited to a subset of language features. To address the prior limitations, my work, **TitanFuzz** [9] demonstrated for the **first time that LLMs can be directly applied to fuzz test real-world software systems like DL libraries and has opened an entirely new area of LLM-based fuzzing.** The key insight is that LLMs can *implicitly* learn both language semantics and DL API constraints from their pre-training data and thus can directly produce valid and diverse fuzzing inputs without *explicitly* modeling and solving the constraints. Experiments showed that TitanFuzz drastically improved code coverage compared to prior tools and uncovered many high-priority bugs in popular DL libraries like PyTorch and TensorFlow. While TitanFuzz demonstrated the power of directly using LLMs for fuzzing, LLMs intrinsically tend to generate *ordinary* programs similar to those seen during training whereas fuzzing favors *unusual* inputs to cover edge cases. To address this, I further developed **FuzzGPT** [10] that teaches LLMs to generate even more unique and interesting inputs by leveraging historical bug-triggering programs that contain rare and valuable code ingredients important for bug finding.

General LLM-based fuzzing. Traditional fuzzers are typically designed for a single language or system, making it difficult to reuse them across different targets. Building on the insights learned from TitanFuzz, I aim to build a general LLM-based fuzzer. To this end, I developed **Fuzz4All** [24] – the first universal fuzzer using LLMs that can generate arbitrary programming languages and target a wide range of systems under test. Fuzz4All leverages LLMs as both an input generator and also a mutation engine to produce diverse and realistic inputs for any target language. To accomplish this, Fuzz4All employs a novel autoprompting technique to automatically produce tailored LLM prompts for each target and the LLM-based fuzzing loop to generate fuzzing inputs. We evaluated Fuzz4All on nine different systems and six languages. Notably, with a simplistic LLM-based implementation, Fuzz4All achieved significantly higher coverage compared to seven

[1]The arXiv version of ChatRepair was posted in January 2023 [28]. To my knowledge, this is the first automated approach to leveraging environmental feedback for LLM-based code generation.

different domain-specific fuzzers (including four award-winning fuzzers) that required high amounts of manual effort to design and implement. In total, my work has found **over 200 bugs or vulnerabilities across popular software such as ML systems (PyTorch, TensorFlow), compilers (C/C++/Java/Go), and even quantum computing framework (Qiskit)**.

Program Synthesis: Advancing Real-World Application

Program synthesis, to turn natural language into code, is widely regarded as the *holy grail* of computer science. Due to the impressive abilities of LLMs, there has been increased use of LLMs to perform program synthesis tasks and tackle software engineering issues. Building on this growing reliance on LLMs for code generation, I work on both evaluating the quality of LLM-synthesized code through benchmark creation as well as techniques to improve LLM synthesis performance on challenging software development problems.

Evaluating program synthesis ability of LLMs. Developers have adopted LLM tools such as Claude Code, OpenAI Codex, and GitHub Copilot to generate code. However, *just how reliable are the programs generated by LLMs?* Prior program synthesis benchmarks rely on manually written tests to cover the diverse ways a program can be used. For more rigorous evaluation of LLM-based synthesis, I developed EvalPlus [12], which combines both LLM and mutation-based test generation to automatically generate vast amounts of high-quality test cases for existing benchmarks. Surprisingly, we found that the performance of all LLMs on the new benchmarks is much lower, demonstrating for the first time that prior benchmarks contain insufficient test cases and do not accurately reflect the true performance of LLMs for code synthesis. EvalPlus has become **one of the default benchmarks to evaluate LLM-based program synthesis, used by hundreds of papers and adopted by multiple big-tech companies (e.g., OpenAI, Google, and Meta)**. EvalPlus has also inspired many follow-up work on more rigorous evaluation of code LLMs. Furthermore, prior code synthesis benchmarks contain a limited amount and variety of problems due to the high manual effort required to create new problems. To tackle this issue, I built **EvoEval** [22], a benchmark suite created by using LLMs to evolve existing benchmarks into new targeted domains for a comprehensive evaluation of LLM program synthesis abilities. Results on 51 different LLMs show a significant drop in performance and shifts in ranking when evaluated on EvoEval, revealing the potential overfitting of existing benchmarks and inspiring more future work for building robust and targeted benchmarks using LLMs. This work has received an **Oral Spotlight Paper Award** at COLM 2024.

Solving real-world software development problems. Recently, LLM agents have been widely deployed to help developers with end-to-end software development tasks, e.g., automatically modifying the repository to resolve an issue. These agents have the ability to autonomously use tools, run commands, observe environmental feedback, and plan for future actions. However, the complexity of these agentic approaches, together with the limited abilities of current LLMs, raises the following question: *Do we really have to employ complex autonomous software agents?* To answer this, I developed **Agentless** [21] – an *agentless* approach to automatically resolve software development issues. Unlike prior agent-based tools, Agentless operates through well-defined stages (such as issue localization, patching, and test generation) inspired by the software engineering literature, without relying on the agent to decide future actions or use complex tools. Agentless demonstrates for the first time that an agentless approach can achieve highly competitive performance and low cost without requiring excessive tools or complex environment interactions. To date, Agentless has attracted **over 1.9K stars on GitHub and has been adopted by top AI companies (e.g., OpenAI, Meta, and DeepSeek) to evaluate their recent models on real-world coding tasks and build their own custom issue-resolution scaffolds**. Furthermore, this work has also been awarded an **ACM SIGSOFT Distinguished Paper** at FSE 2025.

Future Work

Looking forward, my research goal is to improve developer productivity by transforming the software development cycle. I plan to tackle more challenging problems such as automatically synthesizing production-ready software from scratch and detecting critical security issues in large codebases. To achieve this, I aim to design agentic systems that integrate the powerful planning capabilities of LLMs with specialized software engineering tools, enabling the creation of cost-efficient and effective agentic solutions. Ultimately, my vision is to build general-purpose software agents and LLMs that can handle the full spectrum of development tasks, while autonomously and continuously improving themselves throughout the process.

Agents for challenging software tasks. My goal is to build advanced agentic solutions that can tackle challenging software development tasks that require synergistic long-horizon interactions with the environment and access to specialized software engineering tools. I plan to develop agents that can expertly leverage previously underutilized tools such as binary analysis and decompilers to detect malicious vulnerabilities in commercial off-the-shelf binaries, as well as profilers and tracing tools to optimize complex systems.

Furthermore, I want to enable greater multi-agent collaboration since relying on a single large and complex agent to handle many tasks can easily lead to context-window bloating and less reliable performance. I envision a multi-agent system composed of different specialized agents with isolated environments and dedicated tools, coordinated by a main delegation agent that can effectively assign tasks to the individual specialists. This design helps control task complexity, reduces context window of agents, and allows the use of cheaper, more efficient models to solve challenging problems. Ultimately, I aim to build intelligent agentic frameworks that tackle challenging software engineering tasks and advance AI-assisted coding beyond simple automation toward true repository-level intelligence and unlock next-level developer productivity.

Live software agent design. Different tasks require different sets of tools and workflows for a software agent. I recently proposed **Live-SWE-agent** [25], the first *live*, runtime self-evolving software agent capable of autonomously creating its own tools on the fly. Unlike existing self-improving software agents [32, 18] that update themselves through costly offline training on limited benchmarks, Live-SWE-agent updates itself *on the fly* based on the current task and past trajectories. My key insight is that software agents are themselves software systems, and modern agents already possess the intrinsic capability to extend or modify their own behavior at runtime. Built on this idea, Live-SWE-agent is currently the Top-1 open-source agent on both SWE-bench Verified [7] and SWE-Bench Pro [2] (as of November 2025). And this is just the beginning of self-evolving agents, I plan to (1) enable global scaffold modification so agents can change and improve all parts of the scaffold on the fly; (2) support tool-sharing across different problems, enabling evolution not only within a task but also across tasks; and (3) introduce memory-driven evolution that allows agents to transfer insights and behaviors learned from prior attempts to new and unseen scenarios. Beyond runtime adaptation, I envision extending this idea to the training process itself, where instead of learning from a fixed workflow and a static set of tools, the LLM learns to also create new tools and modify the scaffold itself during training. The resulting LLM will have improved reasoning capabilities as the self-evolving training approach provides additional learning signals, allowing the LLM to solve more complex tasks. Moreover, the final trained LLM will be compatible with more runtime agent frameworks since it is able to learn from its own modified scaffolds rather than relying on predefined scaffolds. This adaptability allows it to be more robust and generalize effectively to new scaffold setups and even different tasks.

Improved software agent training. Reinforcement learning (RL) has recently attracted widespread attention for its ability to elicit reasoning capability of LLMs. Software coding agents present a perfect opportunity to develop RL techniques. Meanwhile, existing RL work mainly relies on end-to-end signals, such as if a task has been solved correctly, which can be sparse and costly due to the high amounts of training data required. I believe that we can extract richer signals from the abundance of environmental feedback inherent in the software development process to improve RL training. Furthermore, the software tasks themselves are especially critical for training code models where adding diverse and challenging tasks (e.g., vulnerability detection and production-ready software synthesis) can help the models substantially improve their coding and reasoning capabilities. In addition, I plan to leverage software agents themselves to create new training data for various tasks (e.g., creating new bugs for repair agents to fix and testing agents to find) to further advance model training in a *self-play loop*. Rather than a fixed training routine, my insight is that models under training know best what training data are the most valuable for further improving themselves, and thus can determine autonomously which training tasks and data to generate and learn from on the fly. This strategy not only produces valuable RL training data but also enables the tailored training of agents for more complex tasks where existing training data are limited.

Infinite domain and potential for software research. While my prior research has covered critical parts of the software development cycle, there remain unexplored application domains that I hope to pursue. These include (1) Reliability and security of the LLM ecosystem: ensuring the reliability and security of the environments, LLMs, and agents themselves since they are inherently software systems and can suffer from bugs and vulnerabilities; (2) Intersection of code LLMs and closely relevant areas: including HCI, security, systems, formal methods, and even embodied AI, since code plays a foundational role across all these domains; (3) General LLM reasoning: advancing LLM coding and general reasoning capabilities through massive software-centric training, evaluation, and real-world applications. I believe that there are endless applications of software engineering research with potential to open new directions in how we design, optimize, and reason about intelligent systems. As we move towards an agentic world powered by LLM-driven agents, software itself will become an even more crucial source for training and building the next generation of agents and LLMs. Unlike natural-language text, code is structured, executable, and verifiable, allowing LLMs to learn advanced reasoning and planning skills through RL and environmental interactions. Furthermore, software data is not only complex but also remarkably diverse (as software underlies nearly every aspect of our society), providing a highly condensed knowledge base that we can exploit. I believe that once complex software can be created, tested, and refined autonomously, we can apply these capabilities seamlessly to other computer-driven tasks, paving the path toward more advanced automated reasoning and intelligence.

References

[1] Language server protocol. "https://microsoft.github.io/language-server-protocol/".

[2] Scale AI. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

[3] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo MK Martin, Mukund Raghothaman, Sanjit A Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *2013 Formal Methods in Computer-Aided Design*, 2013.

[4] Earl T Barr, Yuriy Brun, Premkumar Devanbu, Mark Harman, and Federica Sarro. The plastic surgery hypothesis. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014.

[5] Earl T Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE transactions on software engineering*, 2014.

[6] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*.

[7] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Kevin Liu, and Aleksander Madry. Introducing swe-bench verified. *OpenAI Blog*, 2024. https://openai.com/index/introducing-swe-bench-verified/.

[8] Devin, ai software engineer., 2024. "https://www.cognition.ai/introducing-devin".

[9] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *32nd International Symposium on Software Testing and Analysis*, 2023.

[10] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In *Proceedings of the 46th IEEE/ACM international conference on software engineering*, 2024.

[11] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[12] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[13] Fan Long and Martin Rinard. Staged program repair with condition synthesis. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015.

[14] Scott Matteson. Report: Software failure caused $1.7 trillion in financial losses in 2017. *TechRepublic*, 2018. https://www.techrepublic.com/article/report-software-failure-caused-1-7-trillion-in-financial-losses-in-2017/.

[15] Devon H. O'Dell. The debugging mindset. *acmqueue*, 2017. https://queue.acm.org/detail.cfm?id=3068754/.

[16] Replit agent make apps & sites with natural language prompts., 2025. "https://replit.com/ai".

[17] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 2024.

[18] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025.

[19] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[20] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023.

[21] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying llm-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2025.

[22] Chunqiu Steven Xia, Yinlin Deng, and LINGMING ZHANG. Top leaderboard ranking= top coding proficiency, always? evoeval: Evolving coding benchmarks via llm. In *First Conference on Language Modeling*, 2024.

[23] Chunqiu Steven Xia, Yifeng Ding, and Lingming Zhang. The plastic surgery hypothesis in the era of large language models. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering*, 2023.

[24] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4all: Universal fuzzing with large language models. In *46th International Conference on Software Engineering*, 2024.

[25] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-swe-agent: Can software engineering agents self-evolve on the fly? *arXiv preprint*, 2025.

[26] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era of large pre-trained language models. In *Proceedings of the ACM/IEEE 45th International Conference on Software Engineering*, 2023.

[27] Chunqiu Steven Xia and Lingming Zhang. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022.

[28] Chunqiu Steven Xia and Lingming Zhang. Conversational automated program repair. *arXiv preprint arXiv:2301.13246*, 2023.

[29] Chunqiu Steven Xia and Lingming Zhang. Keep the conversation going: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024.

[30] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering*, 2024.

[31] Andreas Zeller, Rahul Gopinath, Marcel Böhme, Gordon Fraser, and Christian Holler. The fuzzing book, 2019.

[32] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.